

M13 SOLO TEACHING BOOK

Delta Lake + Lakehouse | beginner-first, self-contained core teacher

How to use this book

Watch only the assigned lesson visual first, close it, then learn here. The book must make sense even if every external link disappears. Do not open protected Review while learning.

Version baseline

M13 is authored for Apache Spark/PySpark 4.2.0 with delta-spark 4.4.0. Current official documentation overrides older video UI/syntax details. Kafka starts in M14.

DE13-01 - Data Lake vs Warehouse vs Lakehouse

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - What is a data lakehouse?](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can data lake vs warehouse vs lakehouse and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

A data lake stores flexible files cheaply; a warehouse provides managed tables and query-oriented governance; a lakehouse tries to combine open/low-cost storage with table semantics and analytical management. The useful question is not which label sounds modern. Ask what guarantees the consumer needs: transactions, schema, history, reproducibility, access control, performance and tooling.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-01 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Draw three boxes: lake, warehouse, lakehouse. For each, write storage form, table guarantees, typical consumers, and one trade-off. Then place Parquet and Delta correctly: Parquet is a file format; Delta adds table metadata/transaction semantics around data files.

Guided reproduction

Classify five scenarios as lake, warehouse, or lakehouse and justify by guarantees, not brand names.

Independent task - close teaching first

For an IoT telemetry workload, choose lake/warehouse/lakehouse and defend the choice using transactions, history and analytical access.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain data lake vs warehouse vs lakehouse. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed domain: classify a medical-imaging repository architecture and state required table guarantees.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-02 - Why plain files are not tables

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Delta Lake overview](#)

[Delta Lake docs - quick start](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can why plain files are not tables and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

A directory full of Parquet files can hold typed columnar data, but the files alone do not tell concurrent writers which set of files forms the committed table snapshot. A table layer needs metadata that says which files are active, what schema is valid, and which operation produced the current state. That is why “Parquet folder” and “Delta table” are not interchangeable.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-02 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Use the normal simulator output. Compare version_0.json with the input CSV files. Identify the extra table-state information: version, operation, schema and active rows. Explain why a consumer needs a committed snapshot rather than a random directory listing.

Guided reproduction

Inspect base files and version_0 table snapshot; list three guarantees the snapshot metadata adds.

Independent task - close teaching first

Explain why copying a new Parquet file into a table directory is not equivalent to a committed table write.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain why plain files are not tables. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed files: explain why a folder of versioned CSV exports still lacks transactional table semantics.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-03 - Transaction log, ACID and table versions

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Delta Lake overview](#)

[Databricks - Table history](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can transaction log, acid and table versions and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

Delta records table changes in an ordered transaction log. A successful commit creates a new version; readers reconstruct a consistent snapshot from committed actions rather than seeing a half-finished multi-file write. ACID is about transaction behavior, not about making bad business logic correct. Atomicity prevents partial commit visibility; consistency here means table constraints/protocol rules remain valid; isolation keeps concurrent work from exposing intermediate state; durability preserves committed state.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-03 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Open normal/log. Read versions 0 through 3. For each, state operation and row/value control. Show that version numbers are immutable history in the simulator. Then explain what real Delta would add beyond this classroom JSON model.

Guided reproduction

Read commit logs 0–3 and explain atomic committed-version visibility.

Independent task - close teaching first

Given four hypothetical commits, identify what a reader at version 2 should see and what it must not see.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain transaction log, acid and table versions. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed log: reconstruct which snapshot is visible after one failed and two successful commits.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-04 - Create, read, update and delete

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Create and manage Delta tables tutorial](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can create, read, update and delete and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

CRUD describes logical table changes: create/write an initial table, read a snapshot, update matched rows, and delete rows intentionally. In Delta, updates/deletes create new table versions rather than editing a Parquet file in place as if it were a spreadsheet. The transaction layer chooses new active files and records the commit.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-04 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Trace v0→v1→v2→v3 in the normal simulator. Identify the changed and deleted business IDs. Then write pseudocode for CREATE, UPDATE and DELETE in Delta SQL or DeltaTable API without copying the reference implementation.

Guided reproduction

Trace create/read/update/delete transitions and reconcile each version row/value controls.

Independent task - close teaching first

Write your own CRUD pseudocode and the validation after each operation.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain create, read, update and delete. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed state: design create/update/delete checks for inventory records.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-05 - Schema enforcement and schema evolution

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Schema evolution](#)

[Databricks - Clean and validate data](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can schema enforcement and schema evolution and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

Schema enforcement rejects incompatible writes instead of quietly corrupting a table. Schema evolution is a controlled decision to accept a compatible structural change. “Automatically accept every schema change” is not resilience. New nullable columns may be safe in one pipeline, while renaming, dropping or changing types can break consumers. Treat schema as a contract and record who owns changes.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-05 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Compare v1 schema with v2 schema. Explain why adding `quality_band` is a controlled additive change. Then propose one breaking change—such as value becoming free-text—and write the validation that should stop it.

Guided reproduction

Apply one additive schema change and reject one incompatible type-change proposal with a written reason.

Independent task - close teaching first

Classify four schema changes as safe additive, controlled migration, or reject; explain downstream impact.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain schema enforcement and schema evolution. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed schema: decide whether adding nullable `device_model` vs changing amount `STRING→STRUCT` is safe.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-06 - Time travel and version history

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Table history](#)

[Databricks - Delta tutorial](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can time travel and version history and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

Time travel means reading an earlier committed table version or timestamp while its required files are still retained. History is useful for auditing, debugging and recovery, but it is not a forever backup. VACUUM/retention policies can make old snapshots unreadable after old files are physically removed. Never shorten retention just to save space without understanding concurrent jobs and recovery needs.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-06 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Read normal version 0 and current version 2. Compare Ravi/C102 tier and total value. Write a debugging note explaining which version introduced the change and what evidence you used.

Guided reproduction

Compare v0 and v2, identify the change version, and state the retention dependency of time travel.

Independent task - close teaching first

Write a recovery plan using history/time travel without claiming it replaces backup.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain time travel and version history. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed history: identify the correct earlier version for rollback investigation and retention risk.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-07 - MERGE, upserts and idempotent incremental loads

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Learning Journal - Delta Lake MERGE INTO: Upserts, CDC and SCD Patterns \(2026\)](#)

Focused 2026 visual for MERGE, idempotent upserts, CDC and SCD Type 2 patterns.

CURRENT AUTHORITY

[Databricks - MERGE into Delta](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can merge, upserts and idempotent incremental loads and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

MERGE combines source changes with a target using a match key. A common upsert rule is: matched key → update; unseen key → insert. Rerun safety depends on deterministic match logic and source uniqueness. If the source contains multiple rows that match one target row, the result can be ambiguous or fail; deduplicate/resolve the source before merge. “MERGE exists” does not automatically make a pipeline idempotent.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-07 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Use updates.csv and prove one updated ID plus one inserted ID. Run the simulator twice and prove the second run does not add a sixth row. Then inspect duplicate_source.csv and explain why source-key uniqueness is a precondition.

Guided reproduction

Run normal MERGE simulator, rerun it, and prove row/value idempotency; then trigger duplicate-source diagnosis.

Independent task - close teaching first

Design an idempotent upsert contract: key, source uniqueness check, matched action, unmatched action, rerun proof.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain merge, upserts and idempotent incremental loads. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed clinic data: prove MERGE rerun safety and reject duplicate source key P206.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-08 - Slowly Changing Dimension Type 2 with Delta

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Learning Journal - Delta Lake MERGE INTO: Upserts, CDC and SCD Patterns \(2026\)](#)

Focused 2026 visual for MERGE, idempotent upserts, CDC and SCD Type 2 patterns.

CURRENT AUTHORITY

[Databricks - MERGE into Delta](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can slowly changing dimension type 2 with delta and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

SCD Type 2 preserves attribute history instead of overwriting it. When a tracked attribute changes, expire the old dimension row and insert a new current row with a new effective period. Business keys identify the entity across versions; surrogate/version rows represent historical states. The hard part is interval correctness: exactly one current row per business key and no overlapping validity periods.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-08 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Open `scd2_history.json`. Find the changed business key. Verify the old row is closed and the new row is current. State two invariants: one current row per key; historical intervals do not overlap.

Guided reproduction

Validate the generated SCD2 history for one current row per business key and non-overlapping periods.

Independent task - close teaching first

Design SCD2 rows for a customer whose tier changes twice; prove one current row remains.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain slowly changing dimension type 2 with delta. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed employee dimension: create non-overlapping SCD2 intervals after department change.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-09 - Bronze, Silver and Gold layer contracts

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[namaste dataengineer - Medallion Architecture Explained \(2026\)](#)

Short visual for Bronze/Silver/Gold flow; official Databricks medallion docs below are the authority.

CURRENT AUTHORITY

[Databricks - Medallion lakehouse architecture](#)

[Databricks - Delta architecture design](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can bronze, silver and gold layer contracts and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

Medallion architecture is a design pattern, not three magical folders. Bronze preserves raw/replayable source truth plus ingestion metadata. Silver applies reusable cleaning, typing, deduplication and quality rules at a declared grain. Gold publishes business-ready models/aggregates for specific consumers. Every boundary needs controls so rows are not silently lost or duplicated.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-09 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Use normal controls: compare bronze_rows, silver_rows and gold_value. Explain why Bronze can contain both base and update events while Silver represents current-state records. Reconcile Gold total back to Silver before trusting it.

Guided reproduction

Map raw events to Bronze, current clean state to Silver, and region aggregate to Gold; reconcile totals.

Independent task - close teaching first

Define Bronze/Silver/Gold contracts for clinic events, including grain, required fields, quarantine and reconciliation.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain bronze, silver and gold layer contracts. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed support-ticket pipeline: reconcile Bronze events to Silver latest-state to Gold SLA summary.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-10 - File layout, compaction and table maintenance

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Optimize data file layout](#)

[Databricks - VACUUM SQL reference](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can file layout, compaction and table maintenance and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

Logical correctness and physical maintenance are separate. OPTIMIZE/compaction can rewrite many small data files into fewer larger files without changing table meaning. Partition choices should follow access patterns and data size; high-cardinality IDs usually make poor partition columns. VACUUM removes obsolete files after retention rules allow it. It is destructive to historical reachability, so treat it as maintenance with a safety policy.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-10 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Write a maintenance plan for a table partitioned by a low-cardinality date/region field. Include: symptom, metric, compaction action, validation before/after, retention rule, and rollback/time-travel impact.

Guided reproduction

Design a compaction/partition/VACUUM maintenance checklist that preserves logical controls.

Independent task - close teaching first

Choose a partition key for a 2-year order table and reject order_id; define compaction and retention checks.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain file layout, compaction and table maintenance. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed maintenance: diagnose a small-files problem without partitioning by a unique ticket ID.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.

DE13-11 - Lakehouse integration and debugging

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Databricks - Diving into Delta Lake: Unpacking the Transaction Log](#)

Visual foundation for transaction log, ACID, versions, concurrency, time travel and lineage/debugging. Older video; current docs below control modern behavior.

CURRENT AUTHORITY

[Databricks - Delta Lake best practices](#)

[Databricks - Table history](#)

[Delta Lake 4.4.0 release - official Delta Lake blog](#)

[delta-spark 4.4.0 - PyPI](#)

Outcome

By the end of this lesson you can lakehouse integration and debugging and prove the result with saved evidence rather than a plausible screenshot.

Teacher explanation

Debug a lakehouse from contracts outward: source receipt → Bronze landing → Silver validity → Gold reconciliation → transaction/history evidence → physical layout. Do not start by randomly deleting checkpoints or VACUUMing. Use table history and control totals to locate the first bad boundary. Separate business-data defects from schema/protocol failures and from performance/layout problems.

Mental model

Think in four layers: source data → physical files → committed table snapshot → consumer-facing contract. Delta is mainly about making the middle two layers trustworthy and inspectable.

Why data engineers care

A pipeline that produces correct rows once can still be unsafe if retries duplicate records, writers race, schema drifts silently, history disappears unexpectedly, or maintenance changes table meaning. The engineering skill is to preserve both business correctness and operational evidence.

Common beginner traps

- Calling Parquet itself a Delta table.
- Assuming ACID proves business logic is correct.
- Using automatic schema evolution as permission for any upstream change.
- Treating time travel as permanent backup.

DE13-11 - Practice and proof

PART 2 - BUILD → SOLO → VALIDATE → EXPLAIN

Teacher demonstration

Take the unseen practical dataset. Write a six-step incident runbook that identifies the first bad layer, uses row/value controls, inspects history, checks source-key uniqueness before MERGE, and preserves evidence before repair.

Guided reproduction

Debug a seeded mismatch using layer-by-layer controls and table history without destructive cleanup.

Independent task - close teaching first

Diagnose a scenario where Gold revenue drops but Bronze receipt count is normal; state the first three checks in order.

Validate before move-on

Save the input grain, the operation you performed, at least one independent row/value/schema/history control, and the evidence file or command that proves it. If a control differs, diagnose before continuing.

Recover without seeing the answer

1) Restate intended table grain/state. 2) Identify the first version/layer where the control diverges. 3) Check keys/schema before rewriting code. 4) Reveal only the next structural hint. 5) Protected Review is permitted only after a saved attempt.

Explain back

In 60-90 seconds explain lakehouse integration and debugging. Include one failure mode and one validation you would show a teammate.

Fresh transfer if weak

Changed incident: trace a schema-drift failure from Silver back to source and preserve the failed attempt evidence.

Mastered only when

You can repeat the changed task without the lesson/review open, validation passes, and you can defend why the result is trustworthy. A percentage score does not infer mastery.